



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

00/00/2026

Análisis y Diseño de Sistemas 1

Escuela de Ingeniería de Ciencias Y Sistemas

Tutor: Nombre del tutor

UNIDAD 4:

Docker

Anuncios Importantes



Anuncio 1



Anuncio 2



Anuncio 3



Anuncio 4



Anuncio 5

Agenda



TEMA 1



TEMA 2



TEMA 3



TEMA 4



TEMA 5

COMPETENCIA(S) QUE DESARROLLAREMOS



1. El estudiante Analiza conceptos fundamentales de Docker Docker
2. El estudiante Crea imágenes de Docker para empaquetar aplicaciones
3. El estudiante Implementa contenedores Docker para despliegue de servicios
4. El estudiante Utiliza Docker Compose Docker Compose para orquestación de múltiples contenedores



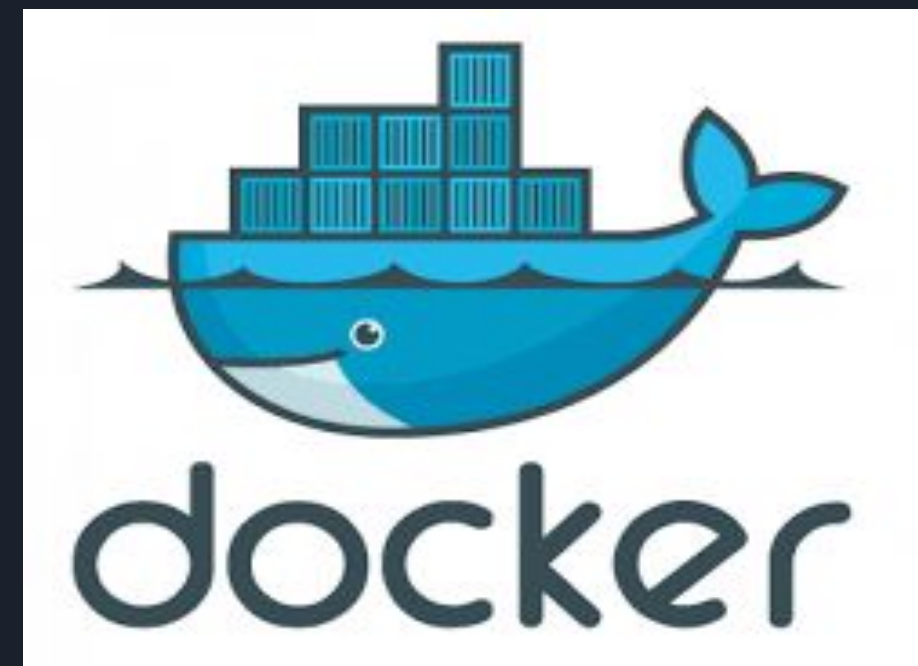
VALOR DE LA SEMANA: Amabilidad

En entornos DevOps con Docker, el trabajo colaborativo es constante: compartir imágenes, revisar Docker Compose de compañeros, reportar errores en contenedores.

La amabilidad facilita la comunicación técnica, hace que las revisiones de código y configuraciones sean más constructivas, y construye la confianza necesaria para que un equipo de desarrollo e infraestructura funcione de manera efectiva.

Docker

Construye una vez.
Ejecuta en cualquier lugar.





El Problema que Resuelve Docker

Desarrollador A	Desarrollador B
Node.js 20	Node.js 18
PostgreSQL 15	PostgreSQL 14
Ubuntu 22.04	Windows 11
Puerto 5432 libre	Puerto 5432 ocupado

"En mi máquina funciona." ← frase más costosa del desarrollo

¿Qué pasa en producción?

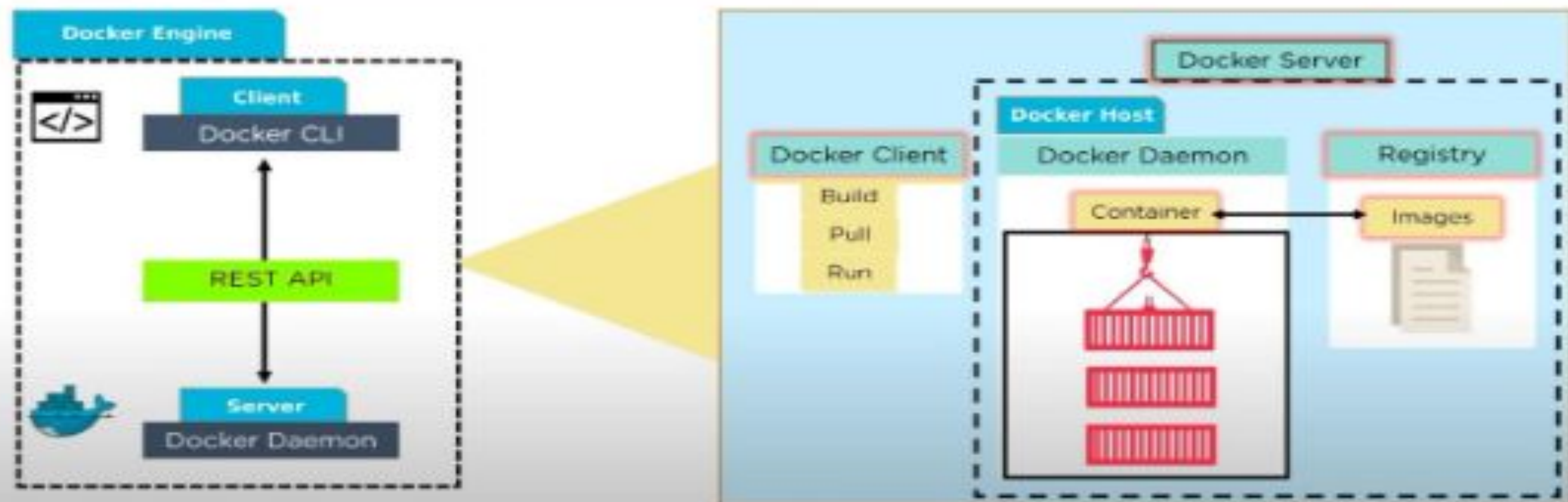
- Servidor tiene Node.js 16, PostgreSQL 13, CentOS 7
- La aplicación explota

SOLUCIÓN: Empaquetar la app con TODO lo que necesita. Eso es Docker.

¿Qué es Docker?

Docker es una plataforma de código abierto para desarrollar, empaquetar y desplegar aplicaciones en contenedores.

Components of Docker





CONCEPTOS FUNDAMENTALES

IMAGEN (Image)

- Plantilla de solo lectura que define el entorno.
- Contiene: OS base, runtime, dependencias, código.
- Como una "foto" del sistema en un momento dado.
- Se comparte y versiona en un Registry.

CONTENEDOR (Container)

- Una instancia ejecutándose de una imagen.
- Aislado del sistema host y de otros contenedores.
- Efímero: puede detenerse, reiniciarse o eliminarse.
- Como "correr" la foto.

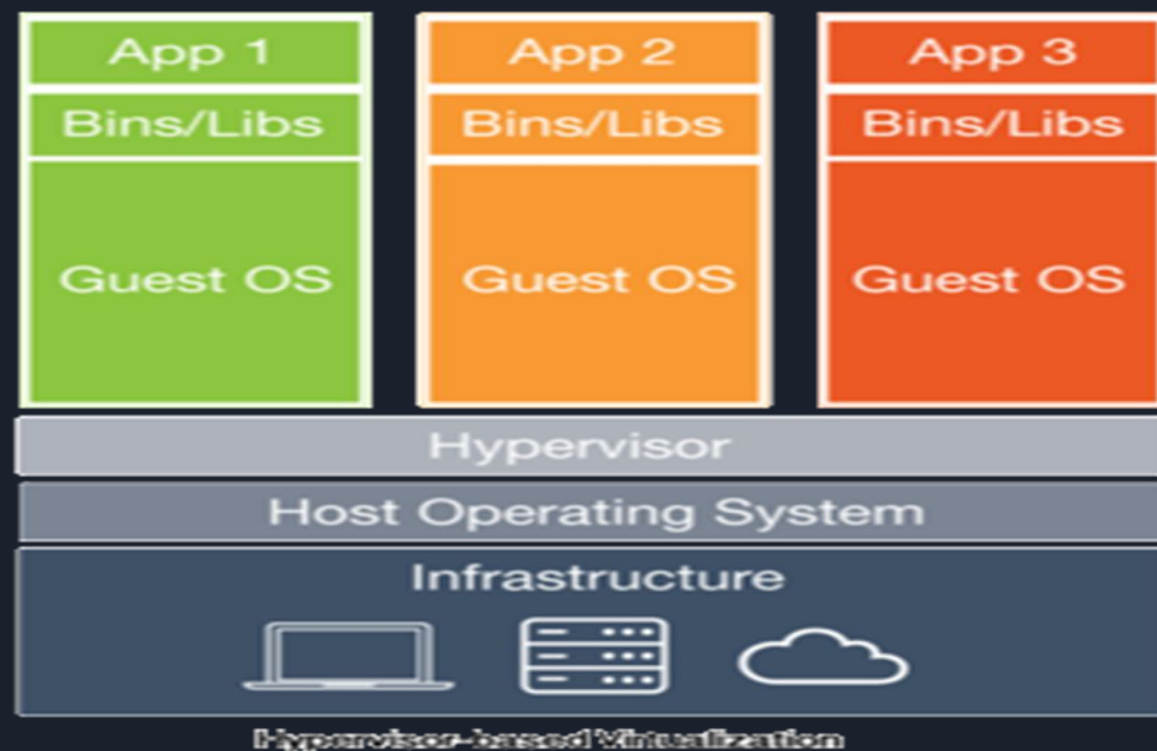
REGISTRY

- Repositorio donde se almacenan y distribuyen imágenes.
- Docker Hub: el registry público oficial.
- También: GitHub Container Registry, AWS ECR, etc.

Imagen : Contenedor = Clase : Objeto

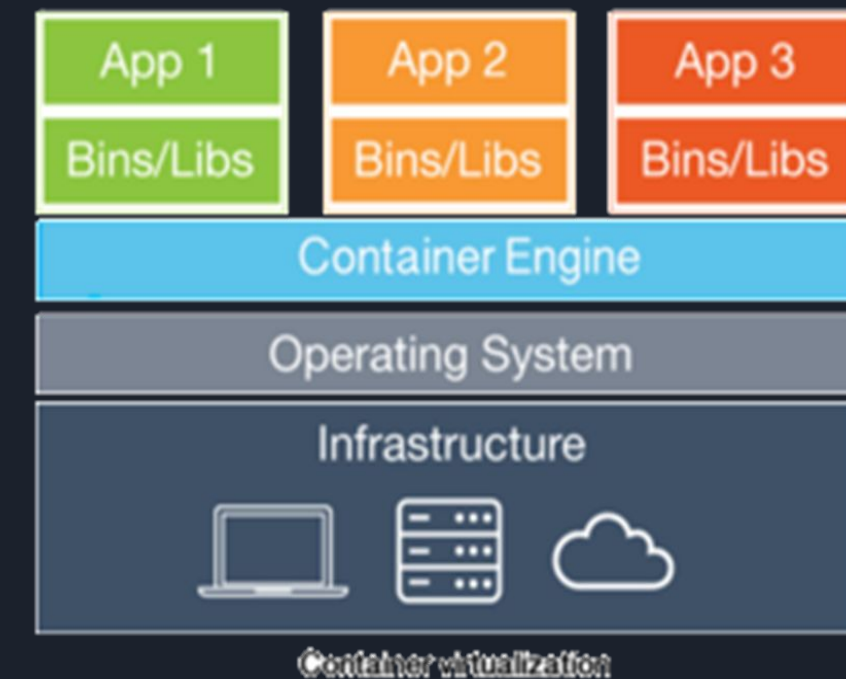
DOCKER vs MÁQUINAS VIRTUALES

Maquina Virtual



- Cada VM carga un OS completo.
- Arranque: minutos.
- Tamaño: GB
- Aislamiento: total.

DOCKER



- Comparte el kernel del host
- Arranque: segundos.
- Tamaño: MB.
- Aislamiento: procesos

Instalación y Comandos

Descargar Docker Desktop desde: <https://docker.com>

Comandos Básicos de Docker

Bash

```
# Comandos Básicos de Docker
docker --version      # Muestra la versión de Docker
docker info           # Muestra información del sistema
docker help           # Muestra la lista de comandos disponibles

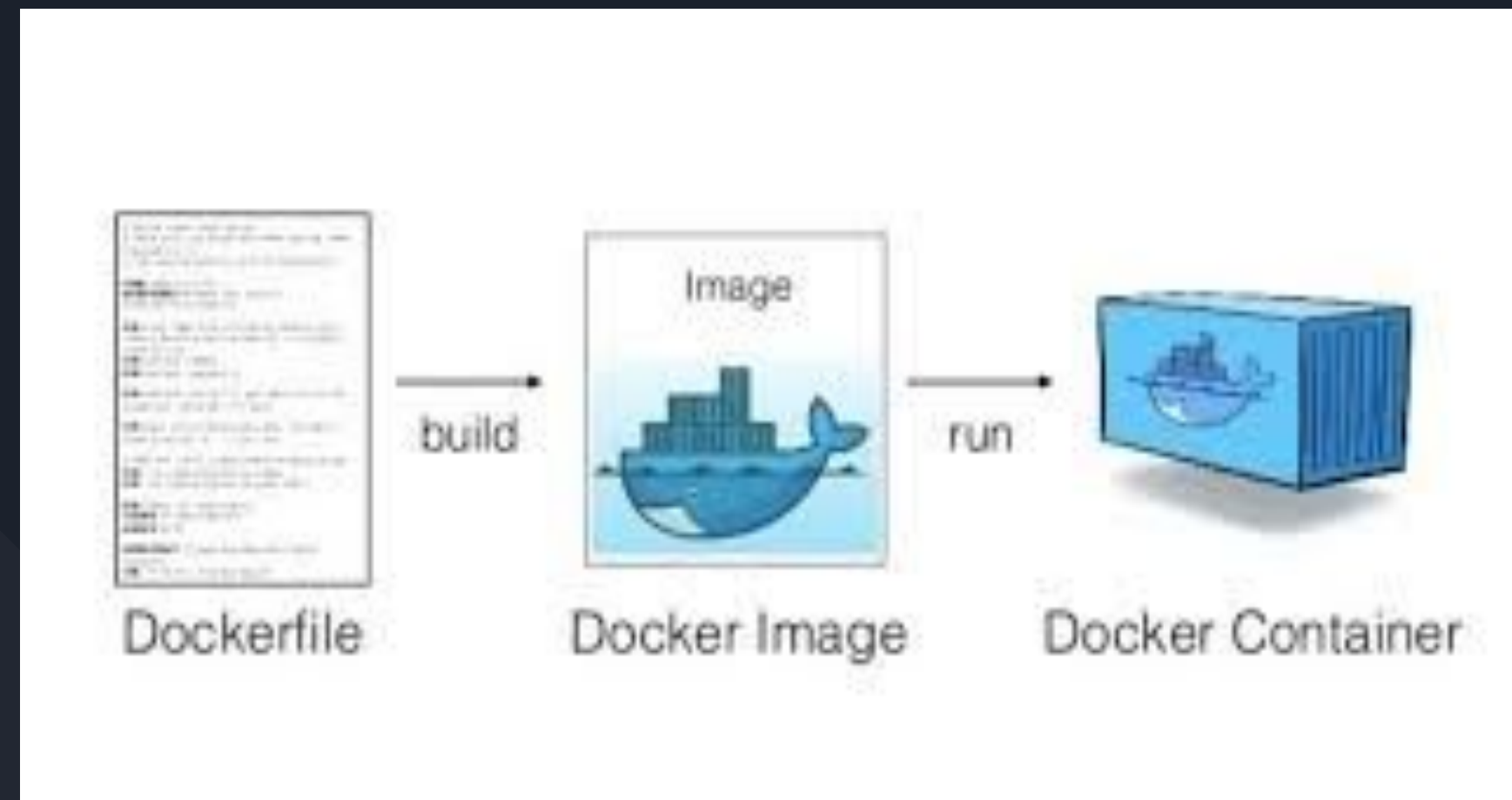
# Trabajando con Imágenes
docker images          # Lista todas las imágenes locales
docker pull <image>    # Descarga una imagen desde Docker Hub
docker rmi <image>     # Elimina una imagen local
docker build -t <name> . # Construye una imagen desde un Dockerfile

# Trabajando con Contenedores
docker ps              # Lista todos los contenedores en ejecución
docker ps -a           # Lista todos los contenedores (incluyendo los detenidos)

docker run <image>      # Crea y ejecuta un contenedor
docker run -it <image>  # Ejecuta un contenedor en modo interactivo
docker run -d <image>   # Ejecuta un contenedor en segundo plano
docker stop <container-id> # Detiene un contenedor en ejecución
docker start <container-id> # Inicia un contenedor que estaba detenido
docker restart <container-id> # Reinicia un contenedor
docker rm <container-id> # Elimina un contenedor detenido
docker exec -it <container-id> bash # Accede al terminal del contenedor en ejecución

# Logs de Contenedores
docker logs <container-id> # Muestra los logs de un contenedor
docker logs -f <container-id> # Muestra los logs en tiempo real
```

CREACIÓN Y GESTIÓN DE IMÁGENES





DOCKERFILE

Receta para construir una imagen

Un archivo de texto con instrucciones paso a paso.
Ejemplo:

```
# Imagen base (OS + runtime)
FROM node:26-alpine

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiar archivos de dependencias primero (caché)
COPY package*.json ./

# Instalar dependencias
RUN npm ci --only=production

# Copiar el resto del código fuente
COPY . .

# Compilar TypeScript
RUN npm run build

# Puerto que expone la aplicación
EXPOSE 3000

# Comando que se ejecuta al iniciar el contenedor
CMD ["node", "dist/server.js"]
```



Instrucciones del Dockerfile

- FROM <imagen>:<tag> Base de la imagen. Siempre primera.
- WORKDIR <ruta> Establece el directorio de trabajo.
- COPY <src> <dst> Copia archivos del host al contenedor.
- ADD <src> <dst> Como COPY pero descomprime .tar.gz.
- RUN <comando> Ejecuta durante el BUILD de la imagen.
- CMD ["cmd","arg"] Comando por defecto al iniciar.
- ENTRYPOINT ["cmd"] Comando fijo (CMD es sus argumentos).
- EXPOSE <puerto> Documenta qué puerto usa la app.
- ENV <nombre>=<valor> Variables de entorno.
- ARG <nombre> Variable solo disponible durante build.
- VOLUME <ruta> Punto de montaje para datos persistentes.
- USER <usuario> Cambia el usuario para los siguientes RUN/CMD.



Construir y Etiquetar Imágenes

Construir desde el directorio actual

```
docker build -t task-api:1.0.0 .
```

Estructura del tag:

- task-api → nombre de la imagen
- 1.0.0 → versión (tag)
- . → contexto de build (directorio actual)

Construir con múltiples tags

```
docker build -t task-api:1.0.0 -t task-api:latest .
```

LISTAR Y ELIMINAR IMÁGENES:

- docker images → Ver todas las imágenes
- docker image inspect task-api → Detalles de una imagen
- docker rmi task-api:1.0.0 → Eliminar por tag
- docker image prune → Eliminar imágenes sin usar

BUENAS PRÁCTICAS DE VERSIONADO:

- task-api:1.0.0 → versión específica (inmutable)
- task-api:1.0 → minor version más reciente
- task-api:latest → última versión estable

.dockerignore y Optimización de Capas

.dockerignore — Excluir archivos del contexto de build

1. Reduce el contexto de build (más rápido)
2. Evita enviar secretos al daemon de Docker
3. Evita sobrescribir node_modules del contenedor

```
1 node_modules/  
2 dist/  
3 .git/  
4 .env  
5 *.log  
6 coverage/  
7 tests/  
8
```

OPTIMIZACIÓN DE CAPAS (orden importa):

- **Bien:**
 - COPY package*.json ./
 - RUN npm ci
 - COPY ..
- **Mal**
 - COPY ..
 - RUN npm ci
 - (cualquier cambio en src/ invalida el caché de npm)

MULTI-STAGE BUILD (imagen final más pequeña):

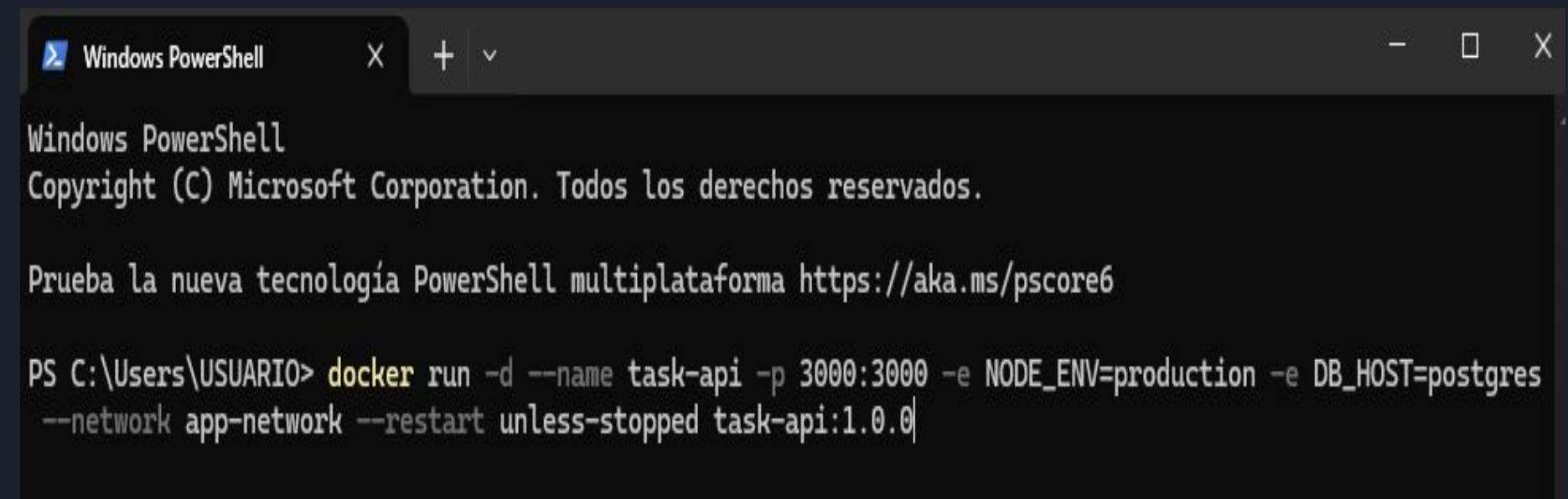
```
1  
2 FROM node:26-alpine AS builder  
3 WORKDIR /app  
4 COPY . .  
5 RUN npm ci && npm run build  
6  
7 FROM node:26-alpine AS production  
8 WORKDIR /app  
9 COPY --from=builder /app/dist ./dist  
10 COPY --from=builder /app/package*.json ./  
11 RUN npm ci --only=production  
12 CMD ["node", "dist/server.js"]  
13 # Resultado: imagen sin devDependencies ni código TypeScript
```

DOCKER RUN — OPCIONES ESENCIALES

`docker run [opciones] <imagen> [comando]`

- `-d, --detach` Ejecutar en background (detached)
- `-p, --publish` Mapear puertos
host:contenedor
- `-e, --env` Variable de entorno
- `--env-file` Archivo con variables de entorno
- `--name` Nombre del contenedor
- `-v, --volume` Montar volumen o directorio
- `--rm` Eliminar contenedor al detenerse
- `--network` Red de Docker a usar
- `-it` Modo interactivo (terminal)
- `--restart` Política de reinicio

Ejemplo:

A screenshot of a Windows PowerShell terminal window. The window title is "Windows PowerShell". The text inside shows the standard PowerShell startup messages: "Copyright (C) Microsoft Corporation. Todos los derechos reservados." and a link to try PowerShell. Below that, a Docker command is entered at the prompt: `PS C:\Users\USUARIO> docker run -d --name task-api -p 3000:3000 -e NODE_ENV=production -e DB_HOST=postgres --network app-network --restart unless-stopped task-api:1.0.0`

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Prueba la nueva tecnología PowerShell multiplataforma https://aka.ms/pscore6

PS C:\Users\USUARIO> docker run -d --name task-api -p 3000:3000 -e NODE_ENV=production -e DB_HOST=postgres
--network app-network --restart unless-stopped task-api:1.0.0
```

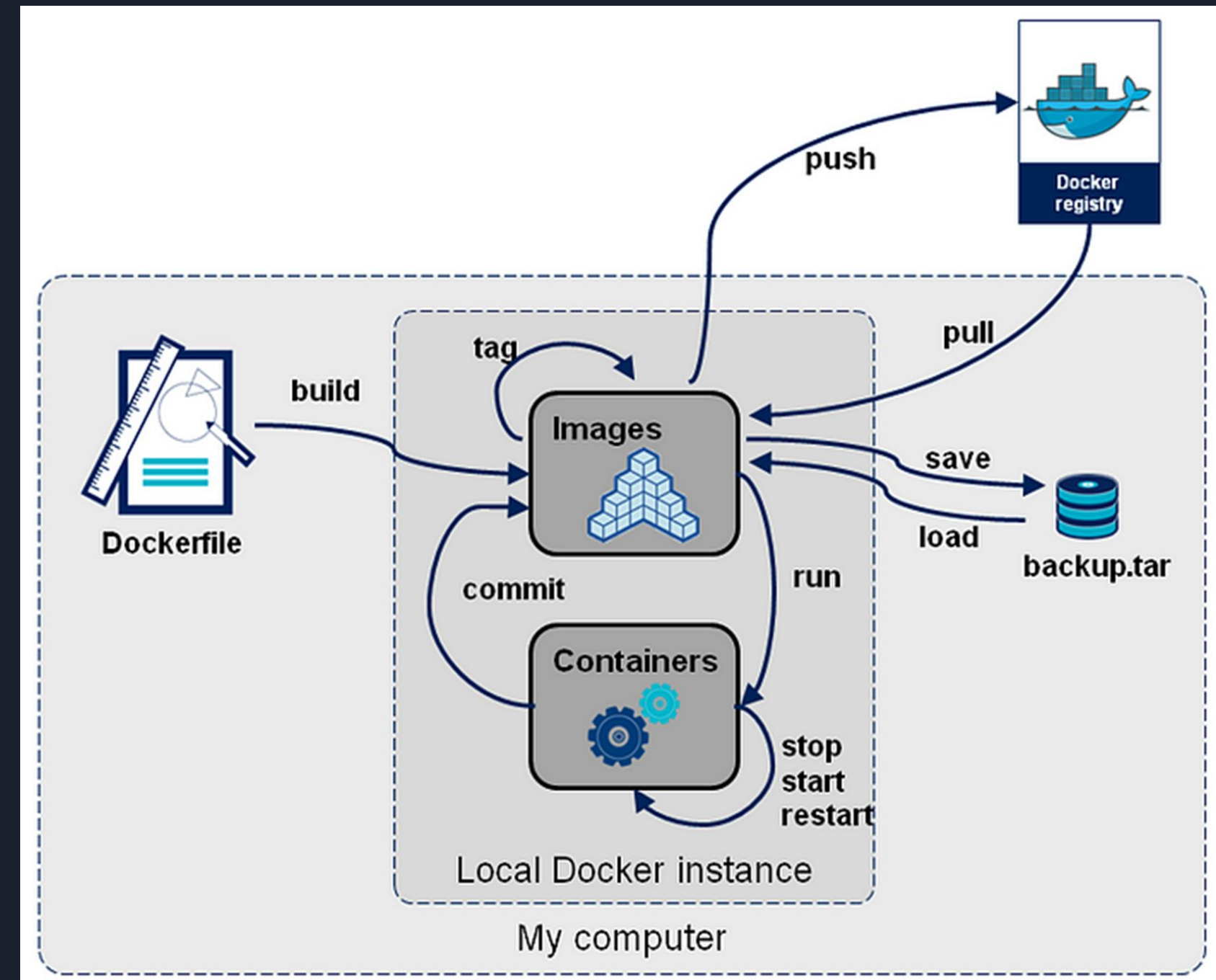

CICLO DE VIDA DEL CONTENEDOR

COMANDOS DE GESTIÓN:

- `docker start <nombre>` → Iniciar contenedor detenido
- `docker stop <nombre>` → Detener gracefully (SIGTERM)
- `docker kill <nombre>` → Detener forzado (SIGKILL)
- `docker restart <nombre>` → Detener y reiniciar
- `docker pause <nombre>` → Pausar (congela el proceso)
- `docker unpause <nombre>` → Reanudar

INSPECCIÓN Y DIAGNÓSTICO:

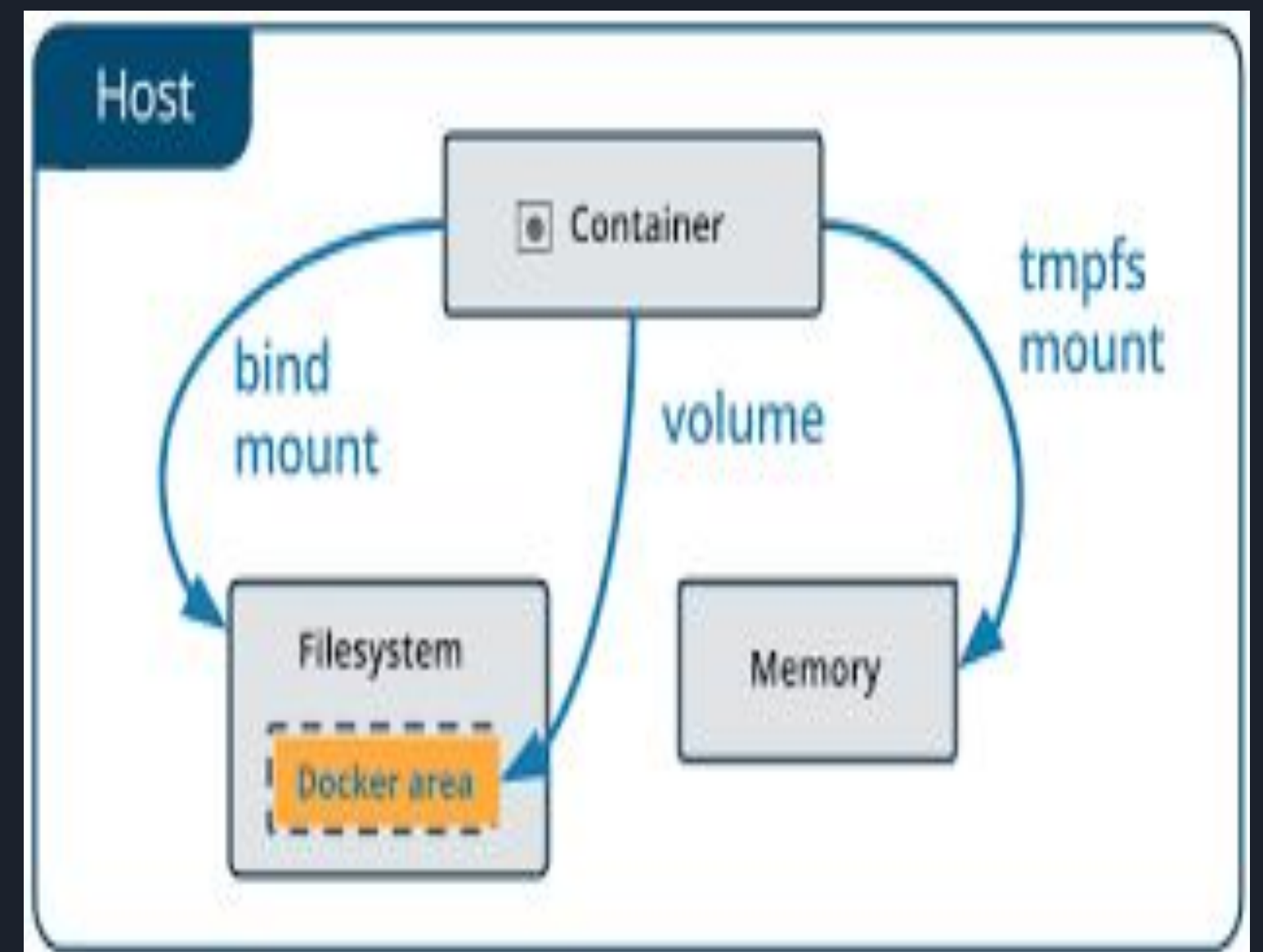
- `docker logs <nombre>` → Ver logs del contenedor
- `docker logs -f <nombre>` → Seguir logs en tiempo real
- `docker stats` → Uso de CPU/RAM en tiempo real
- `docker inspect <nombre>` → Configuración completa en JSON
- `docker exec -it <nombre> sh` → Abrir terminal dentro



VOLÚMENES — DATOS PERSISTENTES

PROBLEMA: Los contenedores son efímeros. Al eliminar un contenedor, sus datos desaparecen.
SOLUCIÓN: Volúmenes

Tipo	Descripción
Named Volume	<code>docker volume create db-data</code> • Gestionado por Docker. • Persiste entre contenedores.
Bind Mount	<code>-v /host/path:/container/path</code> • Sincroniza directorio del host. • Útil en desarrollo (hot reload).
tmpfs Mount	<code>-- tmpfs /app/temp</code> • Solo en memoria, no persiste.



Docker Compose

Docker Compose es una herramienta que permite definir y ejecutar aplicaciones multicontenedor usando un solo archivo YAML (docker-compose.yml).

Comandos Básicos:

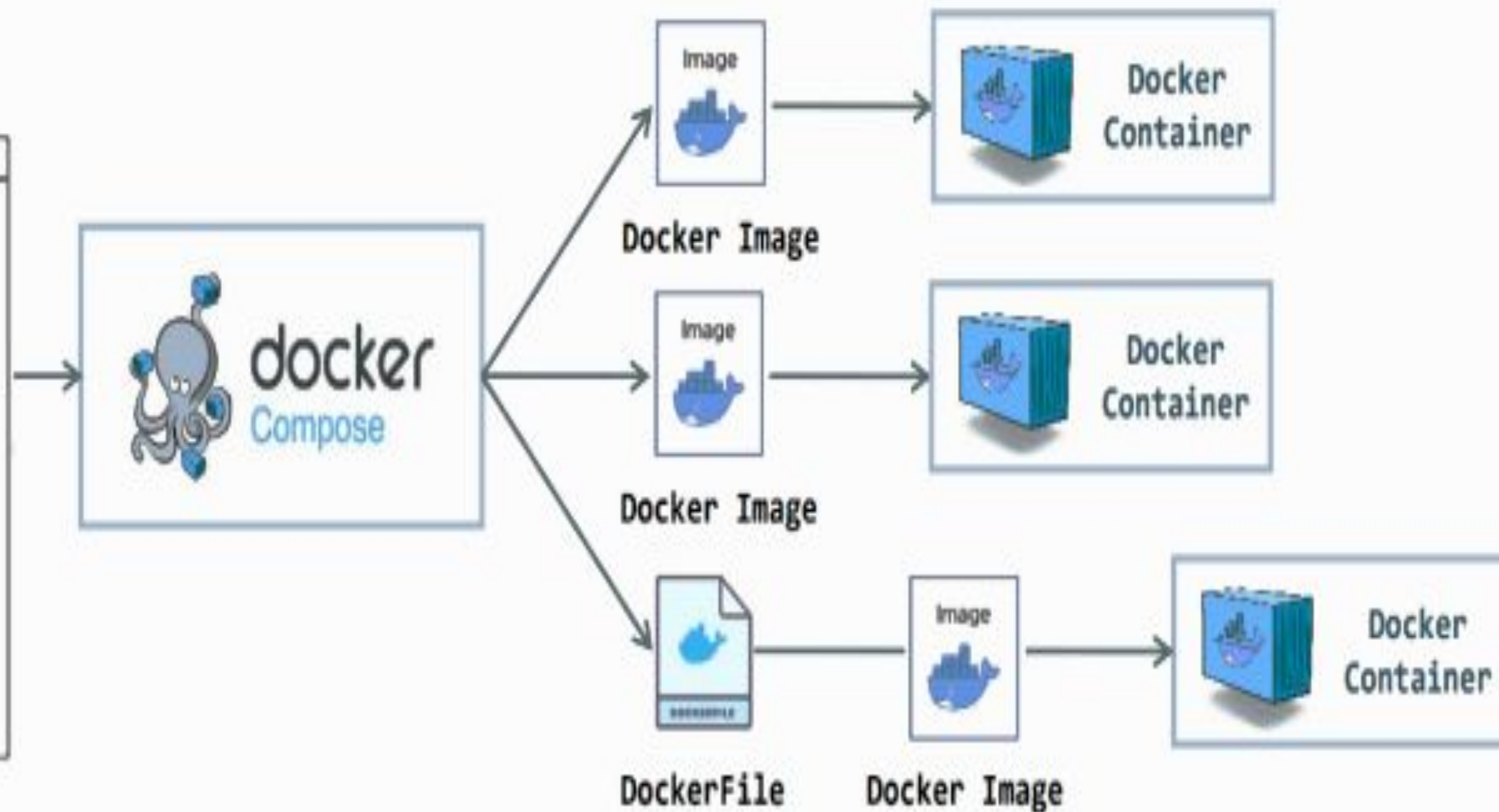
Comando	Función
<code>docker compose up</code>	Levanta servicios
<code>docker compose up -d</code>	Levanta en segundo plano
<code>docker compose down</code>	Detiene y elimina contenedores
<code>docker compose build</code>	Construye imágenes
<code>docker compose ps</code>	Lista contenedores
<code>docker compose logs</code>	Ver logs

Ejemplo docker-compose.yml

```
2  services:
3    db_users:
4      container_name: maria-db-users
5      image: mariadb:latest
6      volumes:
7        - ${USERS_DB}:/var/lib/mysql
8      restart: always
9      ports:
10       - "3307:3306"
11      environment:
12        MYSQL_ROOT_PASSWORD: ${DB_PASSWORD_USERS}
13        MYSQL_DATABASE: ${DB_NAME_USERS}
14      networks:
15        - webapp-users-db-network
16
17    web-app:
18      container_name: webapp
19      build: ./BackEnd
20      command:
21        sh -c "sleep 5 && python manage.py makemigrations &&
22              python manage.py migrate &&
23              gunicorn --bind 0.0.0.0:8001 BackEnd.wsgi:application"
24      ports:
25        - "8001:8001"
26      depends_on:
27        - web-scraper-and-api
28        - db_users
29      restart: always
30      volumes:
31        - ${WEB_APP_DATA}:/app
32      environment:
33        DB_HOST: maria-db-users
34        DB_NAME: ${DB_NAME_USERS}
35        DB_USER: ${DB_USER_USERS}
36        DB_PASSWORD: ${DB_PASSWORD_USERS}
37        DB_PORT: 3306
38        DB_ENGINE: ${DB_ENGINE_USERS}
39        DISPLAY: 98
40      networks:
41        - webapp-scraper-network
42        - webapp-users-db-network
43
```


docker-compose.yml

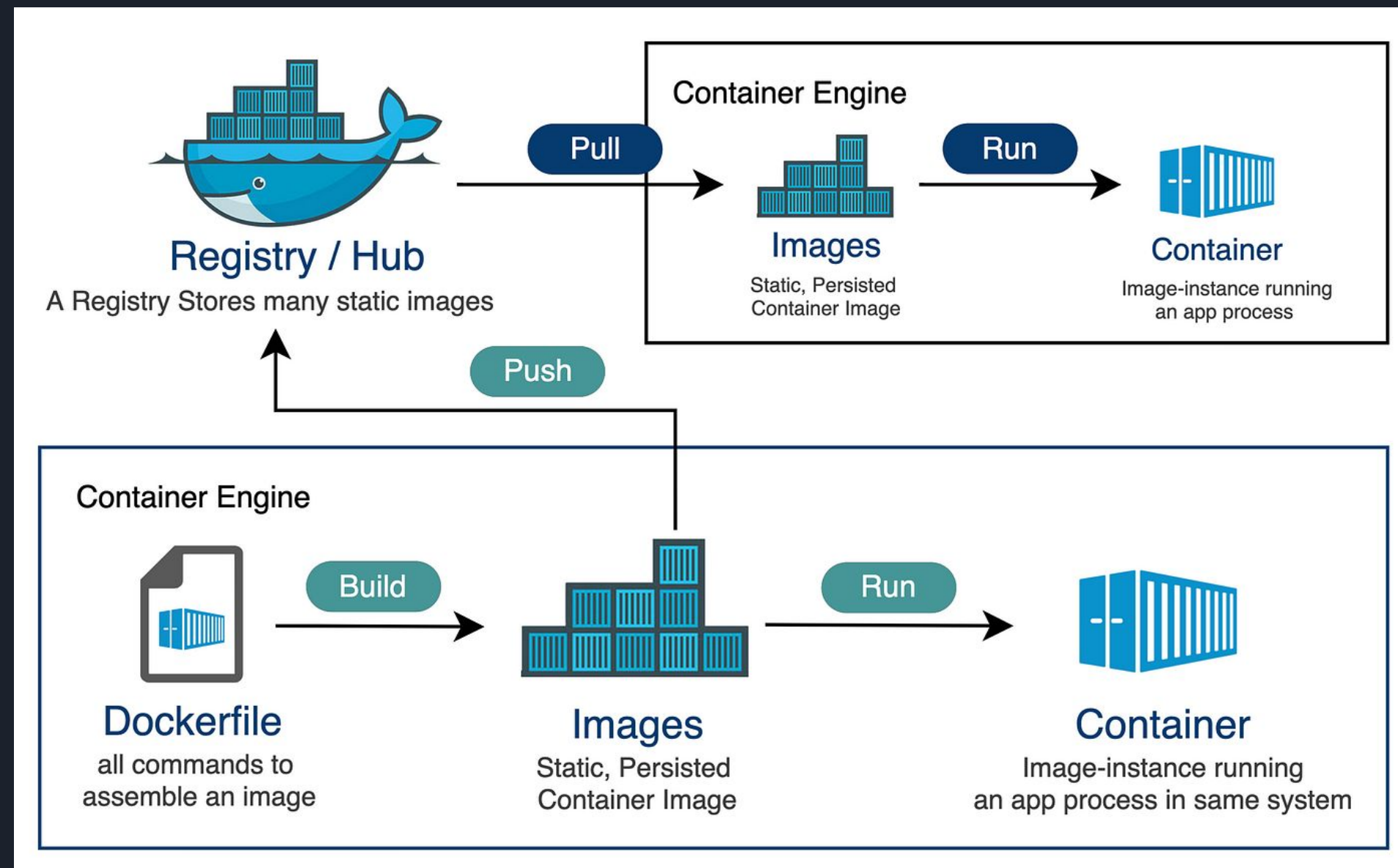
```
version: '3.7'
services:
  db:
    image: mysql:8.0.19
    restart: always
    environment:
      - MYSQL_DATABASE=example
      - MYSQL_ROOT_PASSWORD=password
  app:
    build: app
    restart: always
  web:
    build: web
    restart: always
    ports:
      - 8080
```



¿Qué es un Container Registry?

Servicio para almacenar, versionar y distribuir imágenes Docker.

ES PARA IMÁGENES lo que GitHub es para código fuente.



REGISTRIES PRINCIPALES:

- Docker Hub hub.docker.com (público y gratuito)
- GitHub GHCR ghcr.io (integrado con GitHub Actions)
- AWS ECR *.amazonaws.com (privado, para AWS)
- Google GCR gcr.io (privado, para GCP)
- Registry privado registry:2 (imagen oficial de Docker)



Trabajar con Docker Hub y GHCR

DOCKER HUB — Flujo de trabajo

1. Iniciar sesión
 - a. `$docker login`
2. Etiquetar la imagen con tu usuario
 - a. `$docker tag task-api:1.0.0 tuusuario/task-api:1.0.0`
3. Subir la imagen
 - a. `$docker push tuusuario/task-api:1.0.0`
4. Desde cualquier servidor o equipo:
 - a. `$docker pull tuusuario/task-api:1.0.0`

GITHUB CONTAINER REGISTRY (GHCR) — Más usado en proyectos

1. Login con token de GitHub
 - a. `$ echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --password-stdin`
2. Tag para GHCR
 - a. `$ docker tag task-api:1.0.0 ghcr.io/tuorg/task-api:1.0.0`
3. Push
 - a. `$ docker push ghcr.io/tuorg/task-api:1.0.0`

GitHub Actions: Build y Push Automático

```
1  name: Build and Push Docker Image
2
3  on:
4    push:
5      branches: [main]
6      tags: ['v*']
7
8  jobs:
9    build:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v4
13
14       - name: Log in to GHCR
15         uses: docker/login-action@v3
16         with:
17           registry: ghcr.io
18           username: ${ github.actor }
19           password: ${ secrets.GITHUB_TOKEN }
20
21       - name: Build and push
22         uses: docker/build-push-action@v5
23         with:
24           push: true
25           tags: |
26             ghcr.io/${ github.repository }:latest
27             ghcr.io/${ github.repository }:${ github.sha }
```

- Cada push a main construye y sube la imagen automáticamente.
- Tag con SHA del commit = deploy reproducible y trazable.

CONCEPTOS CLAVE APRENDIDOS

- Docker es una plataforma diseñada para desarrollar, enviar y ejecutar aplicaciones mediante contenedores.
- Las imágenes son la base de los contenedores en Docker.
- Cuando una aplicación depende de múltiples contenedores, como una API, una base de datos y un servicio de caché, Docker Compose simplifica su orquestación
- Docker Hub es el registro público por defecto, pero también existen soluciones privadas

REFERENCIAS

Documentación oficial de Docker

- Docker Desktop: <https://www.docker.com/products/docker-desktop> (docker.com in Bing)
- Documentación general: <https://docs.docker.com/>

Conceptos fundamentales (Imágenes, Contenedores, Registry)

- Docker Hub (registro público): <https://hub.docker.com/>
- GitHub Container Registry (GHCR):
<https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry>
(docs.github.com in Bing)

Dockerfile y construcción de imágenes

- Referencia oficial de Dockerfile: <https://docs.docker.com/engine/reference/builder/> (docs.docker.com in Bing)

Gestión de contenedores y comandos

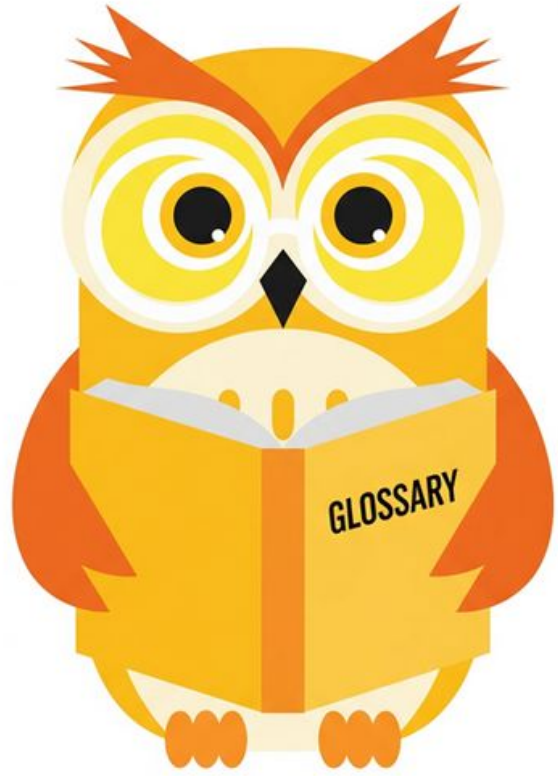
- Comandos básicos de Docker CLI: <https://docs.docker.com/engine/reference/commandline/docker/> (docs.docker.com in Bing)

Docker Compose (orquestración de múltiples contenedores)

- Documentación oficial de Docker Compose: <https://docs.docker.com/compose/>



Ejemplo práctico



Practiquemos lo aprendido



Nombre de la actividad:

Cantidad de participantes:

Doy fe que esta actividad está
planificada en dtt (Sí/No):

Hora de inicio:

Hora de fin:

Duración (min):

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

Recursos

- Se colocará el recordatorio de las actividades según corresponda
- Link del Video: https://youtu.be/limwOve2p6E?si=-W_h8Hn7oiL-6cxy



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA